Punt Labs Launches Beadle

Signed instructions. Declared permissions. Tamperproof audit trail.
Your autonomous agent, on your machine, under your control.

What's real today, as of v0.16.5: Everything from here through the Call to Action — the whole Press Release section — describes Beadle's target end state — a signed, autonomous command-pipeline daemon — written in the Working Backwards convention, as if that end state already shipped. It hasn't. What ships today, and is what a user installing Beadle actually gets, is `beadle-email`: an MCP email server with a four-level PGP trust model and per-contact permissions, installed via a single script (see README.md). The pipeline daemon (`beadle-daemon`) exists as tested, unreleased code; GPG command-signing enforcement — the mechanism this document's headline is named for — is not yet built at all. The Feature Appendix states the exact split.

Press Release

SEATTLE, WA — [target date] — Punt Labs today released Beadle, a free, open-source daemon that turns email into a programmable control plane for an autonomous agent running on your own machine. You send an email; Beadle verifies your identity via PGP or Proton E2E encryption, decomposes your instruction into a pipeline of commands, and executes them — mixing AI reasoning (Claude) with fast CLI tools (biff, jq, git) in a single pipeline. Results arrive as an email reply. Unlike agent frameworks that ask for broad trust, Beadle requires GPG-signed command definitions, enforces per-contact permissions before executing anything, and records every step in a pipeline audit trail. Commands are programs, the daemon is the shell, pipelines are pipes, and GPG signatures are sudo (see FAQ 2).

Problem

Developers today use AI coding assistants interactively: they sit at the keyboard, type a prompt, review the output, and iterate. When the laptop closes, the agent stops. A developer who triggers a two-hour test suite at 6pm must either keep the terminal open until 8pm or accept that a crash at 7pm means starting over in the morning. Delegating multi-step tasks — run tests across three environments, summarize the results, file issues for failures — means cobbling together fragile cron jobs and shell scripts with no authentication, no permissions model, and no audit trail. Existing autonomous agent frameworks like Auto-GPT run locally but offer no cryptographic verification of who authorized which action, no per-command permissions, and no tamperproof record of what happened. The risk is not theoretical: in January 2026, ClawdBot (OpenClaw) went viral, growing from roughly 9,000 to over 60,000 GitHub stars in 72 hours and surpassing 145,000 within about two weeks [1], and was immediately found to have critical vulnerabilities — authentication disabled by default, a one-click remote-code-execution chain, and a spoofed-email attack that exfiltrated API keys [2, 3, 4]. Autonomous agents without enforceable trust boundaries produce real security failures at scale (see FAQ 20).

Solution

Beadle is a background daemon with its own email address, GPG keychain, and a library of GPG-signed YAML command files. Each command declares a runner (Claude for AI reasoning, CLI for deterministic tools), its inputs and outputs via JSON schema, and whether it transforms the pipeline data or passes it through as a side-effect. Commands compose like Unix pipes:

```
summarize | notify-team | reply
```

The owner sends an email to Beadle's address. The daemon verifies the sender's identity (PGP signature or Proton E2E encryption), checks their `x` (execute) permission, maps the instruction to a command pipeline, and executes stages sequentially. A single JSON value — the pipe — flows through the pipeline. Process-mode commands transform it; passthrough-mode commands (notifications, logging) read it without modification. Results arrive as an email reply. Every stage is logged with pipeline ID, command name, runner, duration, and exit status (see FAQ 3).

Customer Quote

> *"I emailed 'summarize this' from my phone on the train and had a structured summary back in my inbox in under two minutes — reasoning done, the headline already broadcast to my team, before I sat down at my desk. I didn't touch a laptop. When my manager later asked whether the agent had actually sent that update or I'd done it myself, I could point at the audit log and answer with certainty instead of a guess."*
>
> — **Alex Petrov**, Senior Platform Engineer, mid-size fintech

Getting Started

Getting started takes three steps. First, install the binary: `curl -fsSL .../install.sh | sh` downloads the Go binary and installs the Claude Code plugin (MCP tools + slash commands). Second, configure your email connection (IMAP/SMTP via Proton Bridge or any provider) and store credentials in your OS keychain. Third, start the daemon: `beadle-daemon run`. Beadle polls your inbox, and when you send an email with "summarize" in the subject, you get a structured summary back within two minutes. No cloud account, no data leaves your machine. Commands are YAML files in `~/.punt-labs/beadle/commands/` — add new capabilities by writing a YAML file and signing it with your GPG key.

Spokesperson Quote

> "At Punt Labs we build on a principle: earn trust to go fast. Every autonomous agent framework asks you to trust it implicitly — give it your credentials, let it run whatever it wants, and hope the logs are honest. We built Beadle on the opposite assumption: the agent has zero authority of its own. Every signed instruction, every declared permission, every tamperproof log entry earns the right to automate more aggressively tomorrow. The signing ceremony isn't overhead — it's the mechanism that compounds trust so you can hand off bigger tasks with confidence. It's the Unix philosophy applied to AI agents: small, composable, auditable tools that do one thing well."
>
> — **Pat Singh**, Punt Labs

Call to Action

Beadle is available now at `github.com/punt-labs/beadle` under the MIT license. Static Go binaries for macOS and Linux (arm64/amd64). Installation requires an IMAP/SMTP server (Proton Bridge recommended for E2E encryption) and GPG for command signing. Documentation, example commands, and the pipeline design document are in the repository.

Frequently Asked Questions

External FAQs

Q1: What is Beadle and who is it for?

Beadle is an open-source agent daemon that receives instructions via email and executes them as multi-stage pipelines. It is for developers who want to delegate tasks to an AI agent running on their own machine — with cryptographic verification of who authorized what. Commands are YAML files that declare a runner (Claude for reasoning, CLI for tools like jq, biff, git), a JSON output schema, and whether they transform the pipeline data or pass it through as a side-effect. The daemon is the shell; commands are programs; GPG signatures are sudo.

Q2: How is Beadle different from Auto-GPT or Claude Code?

Auto-GPT and similar frameworks give an agent broad autonomy with minimal authentication. Claude Code is an interactive assistant that requires the developer at the keyboard (its `--bare` mode enables headless execution but provides no trust model, no command library, and no pipeline orchestration).

Beadle occupies a different point: it has *zero* independent authority. Every command is a GPG-signed YAML file with a declared runner, typed args, and JSON output schema. Pipelines compose explicitly. The trust gate verifies sender identity (PGP or Proton E2E) and per-contact permissions before any execution. CLI commands run in milliseconds with whitelist-only binary resolution and no shell. The trade-off is deliberate: Beadle sacrifices agent autonomy for owner control, auditability, and speed on deterministic operations.

Q3: Why email? Why not a web UI or a CLI?

Email is asynchronous, universal, and already authenticated. Proton Mail provides end-to-end encryption. GPG provides sender verification. Together, they give Beadle a control plane that works from any device — phone, tablet, another computer — without installing a client or opening a port. The concept of email as an agent control channel has engineering precedent: HumanLayer's Agent Control Plane uses email for human-approval of autonomous agent actions [5]. The owner can CC Beadle on a conversation with a colleague, and Beadle can distinguish the owner's signed instructions from the colleague's unsigned replies.

The vision includes local execution via a CLI (`beadle run pipeline.yaml`) for development and testing. Email is meant to be the remote interface, not the only one.

Q4: How do I get started?

This is the target flow, not what's installable today (see FAQ 15 for the gap): run the install script (`curl -fsSL .../install.sh | sh`) to download the Go binary and install the Claude Code plugin. Configure your email connection (IMAP/SMTP) and credentials. Start the daemon with `beadle-daemon run`. Send an email with "summarize" in the subject from a PGP-verified or Proton E2E contact, and receive the summary as a reply. Add commands by writing YAML files in `~/.punt-labs/beadle/commands/`.

Q5: How much does it cost?

Beadle is free and open-source. The recommended path is a Proton Mail paid plan (starting at $3.99/month), because Proton Bridge — the local IMAP/SMTP server this setup is built around — is not available on the free tier [6]. It is not the only path: Beadle connects over standard IMAP/SMTP with no Proton-specific dependency (see FAQ 13), and Fastmail is already used for the project's own signing tests. You also need a GPG keychain (free). If your command pipelines use the Claude interpreter, you will need an Anthropic API key with usage-based billing — unquantified in this document; see FAQ 18 for the corresponding gap on the Punt Labs side. Beadle itself has no cost.

Q6: What happens to my data?

Nothing leaves your machine unless a command document explicitly sends it somewhere. Beadle runs locally as a daemon. With the recommended Proton Bridge setup, email flows through Bridge on your machine to Proton's encrypted servers; with any other IMAP/SMTP provider, it flows to that provider's servers instead (see FAQ 5). Command documents, audit logs, and results stay on your local filesystem. There is no telemetry, no cloud service, and no account with Punt Labs.

Q7: Do I need to know GPG?

Today: not for identity. Beadle resolves who you are through `ethos`, an identity sidecar shared across Punt Labs tools — and if you don't want even that, one plain-text file (`~/.punt-labs/beadle/default-identi` is enough. Signing uses your own GPG key, configured once during `beadle-email install`; you never invoke `gpg` directly, and the key must carry an expiration date — Beadle already rejects non-expiring keys, today, for this signing.

For command signing specifically — the mechanism this document's headline is named for — the honest answer is different: it isn't built, so there is no onboarding flow to evaluate yet. The dedicated `beadle init/sign/verify` commands imagined in earlier drafts of this document were never built; what shipped instead solves identity, not command-signing enforcement. When enforcement ships, it needs its own onboarding design — a Must Do, not yet started (see FAQ 15).

Internal FAQs

Value & Market

Q8: What is the total addressable market?

The primary market is developers who already use AI coding assistants and want unattended execution. LangChain's November–December 2025 survey (1,340 practitioners) found 57.3% have agents in production, up from 51% the year before [7, 8]. The subset who need local-first, authenticated, autonomous execution is narrower — likely tens of thousands of developers who work with sensitive codebases, regulated environments, or privacy-conscious infrastructure (the Proton Mail user base is an imperfect proxy for this audience).

This is not a rigorous bottoms-up estimate — there's no segment count or willingness-to-pay analysis behind it, just a top-down survey stat and an imperfect proxy. We are not claiming a billion-dollar TAM. The honest framing: Beadle targets a niche of security-conscious developers who value auditability over convenience, sized by inference, not arithmetic.

Q9: What evidence do we have that customers want this?

At hypothesis stage, the evidence is almost entirely internal. The full pipeline runs reliably against the author's own mailbox — a real, PGP-verified email triggers a multi-stage pipeline (Claude and CLI runners, JSON schema validation between stages) and returns a result by reply. That validates the mechanics. It does not validate that anyone besides the person who built it wants it: zero external developers have used Beadle (see FAQ 12).

Inferential evidence:

- Auto-GPT has accumulated over 187,000 GitHub stars since its 2023 release, demonstrating strong developer interest in autonomous agents [9].
- Cisco's 2025 Data Privacy Benchmark Study found 64% worry about inadvertently sharing sensitive information via GenAI tools even as 90% see local storage as inherently safer — though the same study found 91% trust global providers for better data protection than they could provide themselves, up 5 points year-over-year. The market is genuinely split, not uniformly local-preferring [10].
- The OWASP GenAI Security Project published a Top 10 for Agentic Applications in 2025, indicating that agent security is a recognized concern, not a fringe worry [11].
- Proton Mail has grown to 100M+ accounts, confirming demand for privacy-focused communication infrastructure [12].
- No existing agent framework offers GPG-based authentication with per-command permissions — the gap is observable, but unvalidated.

Next validation step: see FAQ 12 — it is no longer "generalize the prototype." The prototype has been generalized. What's missing is distribution and the onboarding flow that would let an external developer reach it at all.

Q10: Who are the competitors and why will we win?

Competitor	Strength	Trust model gap
Auto-GPT	Large community, broad agent autonomy	Agent-directed execution; no owner authentication, no declared permissions, no tamperproof audit trail
Claude Code	High-quality code generation, interactive	Operator-attended trust model; requires user at keyboard, cannot delegate unattended authority
Devin	Full autonomous dev environment	Vendor-hosted trust model; execution runs on Cognition's infrastructure, not the owner's machine
Cron + scripts	Simple, well-understood	No trust model; no identity verification, no permission scoping, no composability with LLMs

Beadle does not compete on agent intelligence — it uses existing LLMs (Claude, GPT) as interpreters. The differentiation is the trust model: local-only execution under cryptographic owner control. Cloud providers and SaaS agent platforms cannot replicate this without contradicting their own business model — their revenue depends on hosting execution, managing infrastructure, and retaining custody of the runtime environment. Offering genuine local-only, owner-controlled execution means giving up the recurring billing, telemetry, and platform lock-in that sustain their economics. Competitors can add daemon modes, unattended execution, and even signing features, but they cannot cede control of the execution environment to the owner without dismantling the value capture that funds their products. This is a structural misalignment, not a feature gap.

Q11: What is the customer acquisition strategy?

Open-source distribution via GitHub Releases. Beadle is a CLI tool in the Punt Labs ecosystem (alongside Biff, Quarry, and Vox), which provides cross-project visibility. The secondary purpose — as a showcase for the Jacobson-Cockburn use-case methodology [13] — generates content that attracts developers interested in both the tool and the process.

No paid acquisition. Growth is organic: one launch post (Hacker News, Lobsters), 2–3 blog posts documenting the build process and methodology, and cross-links from other Punt Labs project READMEs. Estimated effort: 20–30 hours of content creation in the first 3 months. If that generates fewer than 100 stars, the positioning missed the audience and content investment should stop.

Q12: What is your next step to validate your vision?

Both Pipeline v1 and Pipeline v2 shipped as tested code since this document's v3.0 draft — CLI runners, process/passthrough data flow, compound CLI steps, and JSON schema validation all merged in `v0.15.0` (2026-04-18). `beadle-email` itself has shipped six releases since, through `v0.16.5`. Building has not been the bottleneck.

Validation has. The external test proposed at v3.0 — 5–10 developers who use Claude Code or Auto-GPT for automation, writing a command YAML, signing it, sending an email, and getting a pipeline result back — has not run, and cannot run yet: `beadle-daemon` is not distributed to

anyone (FAQ 13 covers why), GPG command-signing enforcement is an unimplemented stub, and there is no `beadle init` to abstract key setup.
The next step is therefore not more pipeline features. In order: (1) ship GPG signing enforcement, so the trust model this document is named for is real; (2) build the `beadle init` onboarding flow; (3) distribute `beadle-daemon`; then (4) finally run the external test. Until that test runs, the riskiest assumption in this document — whether the signing ceremony feels like a feature or a burden — is exactly as untested as it was at v3.0.

Technical

Q13: What are the major technical risks?

1. **Proton Bridge reliability for automated use.** Proton Bridge is designed for desktop email clients, not daemons. It may not handle long-running, unattended operation gracefully.
   Why Proton specifically: Beadle's threat model assumes the email channel must protect command integrity end-to-end — from the owner's device to the daemon's mailbox. Proton Mail's end-to-end encryption means even Proton's own servers cannot read command content in transit, which aligns with the local-first, zero-trust design. That said, Beadle connects to Proton Bridge via standard IMAP/SMTP — there is no Proton-specific API or protocol dependency. Supporting other email providers (Fastmail, self-hosted Dovecot, Gmail with app passwords) is a scope decision, not an architectural constraint.
   Design, not yet built: Beadle should monitor Bridge health on a configurable interval and expose status via a status command; on outage it should log, queue pending result deliveries, and retry on a backoff schedule, with a service-manager restart (macOS `LaunchAgent` + `KeepAlive`, or a systemd unit with `Restart=on-failure`) if the daemon itself exits. None of this exists today. `beadle-daemon` has no health monitor, retry queue, status command, or service-manager integration — if Bridge goes down while the daemon is running, email-triggered execution simply stops with no recovery mechanism.
2. **GPG key management complexity.** GPG tooling is notoriously difficult — complexity is consistently cited as its primary operational weakness [14]. If `beadle init` cannot fully abstract key generation and signing, the onboarding friction will kill adoption. This is the hardest unsolved UX problem in the project.
   Alternative evaluated: SSH signing. GitHub and GitLab both support SSH keys for commit signing, and most developers already have an SSH key. However, SSH signing only covers commit-level attestation — it provides no standard for encrypting documents to a recipient, no built-in key expiry or revocation model, and no ecosystem for distributing trust (no equivalent of the OpenPGP Web of Trust or keyservers). Beadle needs signing *and* encryption *and* key lifecycle management; SSH covers only the first.
   What's actually built: identity resolution already works, though not through a bespoke `beadle init` flow — it goes through `ethos`, an identity sidecar shared across Punt Labs tools. A user with no ethos installed at all still gets a working identity from one plaintext file (`~/.punt-labs/beadle/default-identity`). Signing uses the user's own GPG key from the system keyring — not a Beadle-generated one, and not an isolated keychain; only inbound signature *verification* runs in an isolated `GNUPGHOME`, since that's where an untrusted sender's key needs to be quarantined. The signer is configured via a `gpg_signer` field the install flow already prompts for, and Beadle rejects non-expiring keys before every sign. The user never types a `gpg` command — but does need to already own an expiring key and

place its passphrase in the keychain by hand during `beadle-email install`; that residual friction is exactly what `beadle init` exists to remove.
What's still vision: dedicated `beadle init/sign/verify` subcommands. The daemon's command-signature verifier (`VerifySignature`, DES-034) is implemented — canonical signing, isolated verification against a known owner key, GnuPG status-fd classification — and wired into the command-file loader (DES-035), so a configured daemon now rejects an unsigned or tampered command file. Enforcement is opt-in, though: it activates only once an operator sets `owner_handle` or `owner_gpg_key_id` in `daemon.json`, and there is still no `beadle init` onboarding to make that step easy or a dedicated signing tool for a new user to sign their own command files with — that onboarding gap, not the verification or wiring, is now the higher-priority remaining piece (see FAQ 12).

3. **Claude interpreter sandboxing.** When Beadle invokes Claude as a command interpreter, the LLM may attempt actions outside the command's declared permissions — or a message's *content*, not just its sender, could carry a prompt injection, exactly the vector Snyk demonstrated against ClawdBot (see FAQ 20). *Mitigation, already built:* workers spawn via `claude -p -bare`, which skips CLAUDE.md, hooks, plugins, and auto-memory; an adversarial system prompt frames every worker invocation; the worker's `PATH` is restricted to the Claude binary directory plus `/usr/bin`; and the daemon withholds its own credentials from the worker's environment. *Still open:* permission preflighting is authorization bookkeeping, not a content firewall — it establishes who sent a command, not what a message's body asks the interpreter to do once inside its declared scope, and the worker still runs with the real `$HOME` and broad tool access (`Bash`, `Edit`, `Write`). There is no container-level isolation (network namespace, seccomp) yet.

Q14: What dependencies exist on other teams or systems?

- **Proton Bridge** — open-source Go codebase, maintained by Proton AG [6]. Beadle uses it as a standard IMAP/SMTP client. No API dependency beyond the mail protocol.
- **Anthropic API** — required only when using the Claude interpreter. Usage-based billing, no contractual dependency.
- **GnuPG** — the GPG implementation. Mature, widely deployed, maintained by the GnuPG Project. RFC 9580 (OpenPGP, 2024) provides the current standard [15].

No internal Punt Labs dependencies. Beadle is a standalone project.

Q15: What is the estimated development timeline?

This is a hypothesis-stage document. The purpose of Working Backwards is to validate *whether* to build, not to plan the schedule. Detailed hour estimates and calendar targets are premature — and doubly so when AI-augmented development makes historical velocity assumptions unreliable. What matters at this stage is delivery order and cut discipline.

Phased delivery order:

1. **Foundation (shipped, distributed)** — MCP email server with 21 always-on tools plus 2 poll tools gated on config, four-level PGP trust model, per-contact rwx permissions, multi-identity via ethos (with a plain-file fallback that needs no ethos at all), credential isolation (OS keychain). Released as `beadle-email`, currently `v0.16.5`. This is the only layer any user can actually install today.
2. **Daemon + Pipeline v1 (built, not distributed)** — Background daemon with IMAP poller, trust gate (PGP verified + Proton E2E), x-bit enforcement, pipeline orchestrator (command YAML loader, regex planner, sequential executor, auto-reply, crash-safe persistence), worker

spawner via `claude -p -bare` with prompt-level adversarial hardening (see FAQ 13 for what that does and doesn't cover). Tested and e2e verified against a real mailbox, but `make dist` and `install.sh` do not build or ship it.

3. **Pipeline v2 (built, not distributed)** — CLI runners (execute tools in milliseconds instead of a full Claude session), the process/passthrough data-flow model, compound CLI steps (jq | biff without a shell), JSON pipe with schema validation, runner-conditional command validation. Shipped as tested code in `v0.15.0` (2026-04-18) — the same distribution gap as Pipeline v1 applies.
4. **Future (not built)** — GPG command-signing enforcement (`VerifySignature` is implemented, DES-034, but not wired into the command-file loader — see FAQ 12), the `beadle init` onboarding flow, daemon distribution, an LLM planner (replace regex with Claude decomposition), multi-channel triggers (biff chat, webhooks), an internal scheduler, key rotation management.

Cut order: Future items are cut first, but the first three — signing enforcement, `beadle init`, and distribution — are what stand between this document's own thesis and a product a user can actually evaluate. Everything after those three is genuinely deferrable; those three are not. The kill switch is adoption-based: see FAQ 19.

Q16: What is the scaling story?

Beadle is a single-user daemon. "Scaling" means long-running operation over months and years, not multi-tenant growth.

- **Audit log growth.** The log is append-only and unbounded until the owner manually clears it. A developer running 10 pipelines per day generates roughly 50–100 KB of log per day. At one year, that is approximately 20–40 MB — manageable on any modern filesystem. The risk is neglect, not size.
- **Command library size.** Commands are loaded into memory only during preflight and execution, then discarded. A library of hundreds of signed command documents on disk has no runtime cost until invoked.
- **Proton Bridge longevity.** The daemon depends on Bridge staying online for email-triggered execution. Bridge was not designed for months of continuous uptime. *Design, not yet built:* the mitigation should be operational — Beadle should check Bridge health on a configurable interval, a service manager should restart both Beadle and Bridge on failure, and pending result deliveries should queue and retry once Bridge recovers. None of this exists today (see FAQ 13, Feature 17); an internal scheduler that would let non-email-triggered pipelines run independently of Bridge is also unbuilt (Feature 19).

The honest constraint: Beadle does not scale to teams, organizations, or multi-machine deployment. It is a personal agent for one person on one machine. That is a design choice, not a limitation to fix later.

Business

Q17: What is the revenue model?

None. Beadle is free and open-source. The strategic value is threefold: (1) it demonstrates the Punt Labs engineering methodology in a real, useful product; (2) it grows the Punt Labs open-source ecosystem alongside Biff, Quarry, and TTS; (3) it serves as the reference implementation for a future punt-labs project on use-case methodology, generating content and credibility.

Q18: What does the P&L look like at steady state?

Beadle has no revenue, so the P&L is cost-only. The primary cost is developer-hours. Specific *development*-hour estimates are not attempted at hypothesis stage — the purpose of this document is to validate whether to build, not to plan the schedule — but the cost categories are known.

Cost	Notes
Development (primary)	Solo developer. Four phases described in FAQ 15. The dominant cost and the binding constraint on opportunity cost.
Ongoing maintenance	Bug fixes, dependency updates, community PRs. Modest but nonzero indefinitely.
Proton Mail (paid plan)	$48/year. Personal use; already paid for other purposes.
Infrastructure	$0. No servers; runs on developer's machine.

The opportunity cost is the binding constraint: development time spent on Beadle is time not spent on other Punt Labs projects (Quarry features, Biff improvements, LangLearn ports). This cost is governed by a kill switch tied to the metrics in FAQ 19: if, at the 6-month mark after public release, Beadle has fewer than 100 GitHub stars *or* fewer than 10 weekly binary downloads, the project enters archival mode (security patches only, no new feature work). This is a hard threshold, not a judgment call — below either number means the positioning missed the audience and further development hours are not justified.

Where the clock actually stands: first public release (`v0.1.0`) was 2026-03-13. The 6-month mark is 2026-09-13 — about two weeks from this revision's date. The current star count is 3. That is not "below the 100-star failure threshold"; it is roughly 3% of it. The kill switch as originally written also measures the wrong thing: `beadle-daemon`, the product this document is about, isn't distributed (see FAQ 15), so "weekly binary downloads" can only mean `beadle-email` downloads — a proxy for interest in the email MCP server, not in the signed-pipeline-daemon vision.

Q19: What are the key metrics and how will we measure success?

Metric	Target (6 months)	Decision threshold
GitHub stars	500+	Below 100 after launch suggests the positioning missed the audience
Weekly `beadle-email` downloads	50+	Below 10 suggests the email MCP server, specifically, is not useful enough to keep
External pipeline test completion	8/10 users complete first pipeline	Below 5/10 signals fatal usability friction — untested; blocked on FAQ 12

These metrics don't measure the actual product. Star count and `beadle-email` downloads track interest in the email MCP server Punt Labs already ships. They say nothing about the signed command-pipeline daemon — the thing this document's headline, press release, and competitive positioning are all about — because that daemon has never been distributed to

anyone. A revised kill switch needs either its own instrumentation for the daemon once it ships, or an honest statement that the current metrics are a proxy for a different, adjacent hypothesis ("does an MCP email server with a trust model have users?") than the one this document exists to test.

Recommendation: do not run the kill switch as originally written — it would fire on a metric that was never measuring the right thing. Re-baseline the clock to `beadle-daemon`'s actual distribution date (see Feature 15), and hold the Must Do items to their own accountability instead: if signing enforcement, `beadle init`, and daemon distribution aren't shipped within one quarter of this revision, that — not a star count — is the signal to archive.

Where the clock actually stands, as of this revision: 3 GitHub stars against a first-release date of 2026-03-13, with the 6-month mark landing 2026-09-13 — about two weeks out. Whatever the right kill-switch design turns out to be, the current trajectory does not clear even the "below 100 means it missed" failure bar, let alone the 500-star target.

Pre-mortem — what failure looks like: written at v3.0, imagining March 2027. The failure condition it describes could now arrive well before then, for the reason stated above rather than the one originally imagined.

It is March 2027. Beadle has 200 GitHub stars, 8 binary downloads per week, and the prototype test showed 3/10 users abandoned during GPG key setup. The cryptographic trust model — signed commands, declared permissions, tamperproof audit trail — resonated in blog posts but not in practice. Developers found the signing ceremony too heavy for personal automation tasks where they already trust themselves. The security posture Beadle enforces felt like overhead, not protection. The project becomes a methodology showcase only, with no active user community, and enters archival maintenance.

Q20: Why now? What has changed?

Four shifts make this the right time:

1. **AI agents went mainstream in 2024–2025.** LangChain's 2024 survey found 51% production deployment [8]; its November–December 2025 follow-up (1,340 practitioners) puts that at 57.3%, still climbing [7]. Developers are past the question of "should I use an agent?" and into "how do I trust one?"
2. **OpenPGP modernized.** RFC 9580 (2024) updated the OpenPGP standard for the first time in decades, adding modern cryptographic algorithms and improving key management [15]. GPG tooling is better than it was even two years ago.
3. **OWASP formalized agent security risks.** The OWASP Top 10 for Agentic Applications (2025) legitimized the concern that autonomous agents need security models, not just capability models [11]. Beadle's trust model is a direct response.
4. **ClawdBot proved the threat model in production.** In January 2026, ClawdBot (also known as OpenClaw) went viral, growing from roughly 9,000 to over 60,000 GitHub stars in 72 hours and surpassing 145,000 within about two weeks [1]. Within days, security researchers found it riddled with critical vulnerabilities. CVE-2026-25253 (CVSS 8.8) was a one-click RCE: the Control UI trusted an unvalidated `gatewayUrl` query parameter, auto-connected a WebSocket to it, and leaked the operator's gateway auth token in the connect payload — enabling cross-site WebSocket hijacking, sandbox disable, and Docker escape to full host compromise from a single malicious link [3, 16]. CVE-2026-24763 was a separate, authenticated command-injection flaw via unsafe `PATH` handling in Docker sandbox execution [17]. Separately again, a February 2026 audit found 341 malicious skills among the 2,857 published on OpenClaw's skill marketplace (11.9% of the registry), most traced to one

campaign distributing infostealer malware to an estimated 300,000 users [18]. Snyk demonstrated that a spoofed email could trigger a shell command that exfiltrated `clawdbot.json` containing API keys [4]. Palo Alto Networks' Unit 42 concluded that OpenClaw "maps to all 10 OWASP risks for agentic applications" and lacked "enforceable trust boundaries between untrusted inputs and high-privilege reasoning" [19]. Kaspersky reported that authentication was disabled by default [2].

Of these, only the Snyk finding is squarely the failure mode Beadle's trust model addresses: a spoofed sender triggering unauthenticated execution is exactly what per-contact permissions and PGP sender-verification prevent. The gateway RCE was a browser-mediated credential-theft chain and the skill-marketplace compromise was a supply-chain vetting failure; Beadle's design doesn't claim to address either. The honest framing is narrower than "exactly the failure mode Beadle prevents": OpenClaw proved that shipping an agent with *any* weak trust boundary is dangerous at scale, and Beadle's differentiator is the one boundary — sender authentication — that incident specifically validates.

Risk Assessment

Risk	Rating	Assessment
Value	Medium	Developer interest in autonomous agents is proven (Auto-GPT stars, LangChain survey). Whether the cryptographic trust model — signed commands, declared permissions, tamperproof audit trail — resonates with developers who already trust their own machines is the core untested assumption. The security posture may feel like overhead for personal automation. Validation plan: prototype test with 5–10 developers (see FAQ 12).
Usability	High	GPG abstraction is the hardest unsolved UX problem in the project. The `beadle init` command must generate keys and produce an identity file without ever exposing raw GPG semantics to the user. SSH signing was evaluated and rejected because it lacks encryption and key lifecycle primitives (see FAQ 13). If the abstraction leaks — if any step requires the user to understand keyrings, trust models, or `gpg` flags — onboarding friction will kill the product. The signing ceremony (sign every command, declare every permission) is the trust model's core value but also its primary usability risk: developers may perceive it as ceremony, not safety.
Feasibility	Medium	Pipeline v1 and v2 are both built and e2e verified: email arrival triggers a multi-stage pipeline (Claude and CLI runners, JSON schema validation) and delivers results to the sender. The trust gate, worker spawner, pipeline executor, and crash-safe persistence all work against a real mailbox. What's untested is the opposite of "more code": Proton Bridge has run continuously for weeks, not the months this document's own scaling FAQ (FAQ 16) says is the real bar, and the primary Usability mitigation (`beadle init`) is entirely unwritten. The hardest remaining work is distribution and onboarding, not pipeline mechanics.
Viability	Medium	No revenue requirement, no infrastructure cost beyond the developer's own machine — on that axis the risk is genuinely low. But the stated kill switch measures the wrong artifact: "weekly binary downloads" can only mean `beadle-email` downloads, since that's the only binary distributed, and a download of the email MCP server says nothing about whether the signed-pipeline-daemon vision this document is named for has any pull. The 6-month-post-release clock lands 2026-09-13 — see FAQ 18 and FAQ 19 for the current numbers and the recommendation to re-baseline it.

Feature Appendix

Defines the scope boundary for the product. Every feature is explicitly categorized to prevent scope creep and force prioritization decisions upfront. This revision deliberately splits the usual Must/Should/Won't Do into five categories instead of three — Shipped, Built-Not-Distributed, and Must Do stand in for what would otherwise be a single undifferentiated Must Do, because the distribution gap between "exists in the repo" and "a user can get it" is this document's central finding (see the scope note at the top).

Shipped (distributed to users)

Features a user gets by installing Beadle today. This is the whole shipped product — `beadle-email` — and nothing else in this appendix reaches a user without it.

- **F1.** **MCP email server** — 21 always-on tools plus 2 poll tools gated on config — list, search, read, send, reply, mark, move (single + batch), attachments, verify signatures, MIME inspection, trust classification, address book, identity switching, inbox polling
- **F2.** **Four-level transport trust** — trusted (Proton E2E), verified (PGP), untrusted (bad PGP), unverified (none). Inline PGP verification during message listing
- **F3.** **Per-contact rwx permissions** — read, write, execute per contact per identity
- **F4.** **Multi-identity via ethos** — identity resolved per-request from ethos sidecar, with a plain-file fallback that needs no ethos install at all. Repo-local config, global fallback, mid-session switching
- **F5.** **Credential isolation** — secrets resolved at runtime from OS keychain (macOS Keychain, Linux pass/libsecret), secret files, or env vars. Never stored in config

Built, Not Distributed

Real, tested code that no installed user can reach — `make dist` and `install.sh` build and ship `beadle-email` only. Reaching users is gated on the Must Do items below, not on more building.

- **F6.** **Pipeline v1** — background daemon with IMAP poller, trust gate, command YAML loader with typed args, regex planner, sequential executor, auto-reply as terminal stage, crash-safe pipeline persistence, worker spawner via `claude -p -bare` with adversarial isolation
- **F7.** **Email trigger via Proton Bridge** — receive instructions by email, verify sender identity, execute pipeline, return results by email
- **F8.** **CLI runner** — execute tools (biff, jq, git, ethos) directly via `exec.Command` in milliseconds. Whitelist-only binary resolution. No shell, no LLM overhead for deterministic operations
- **F9.** **Process/passthrough modes** — commands declare whether they transform the pipe (process) or read it as a side-effect (passthrough). Notifications don't destroy pipeline data
- **F10.** **Compound CLI steps** — chain binaries within a single command (jq | biff) via goroutine-per-step with io.Pipe. No shell invoked
- **F11.** **JSON pipe with schema validation** — structured data flows between stages. Commands declare output schemas. Daemon validates JSON between process-mode stages
- **F12.** **Runner-conditional validation** — prompt and budget required for Claude commands only. Binary and steps for CLI only. Typed args for both

Must Do

What has to be true before this is a product a user can evaluate, not just code that passes tests. Launch-blocking.

- **F13.** **GPG command signing enforcement** — verify GPG signatures on command YAML files at daemon startup. Reject unsigned or tampered files. The verifier is implemented (`VerifySignature`, DES-034) and wired into the daemon's command loader (DES-035) — this is the mechanism the document's headline is named for. It is opt-in, not yet the default: an operator sets `owner_handle` or `owner_gpg_key_id` in a new `daemon.json` to turn enforcement on; a daemon that has not configured either still loads command files unsigned, since no signing tool exists yet for a new user to sign them with
- **F14.** **`beadle init` onboarding** — abstract identity and signing setup so a new user never touches raw `gpg`. The Usability risk in this document is rated High specifically because this doesn't exist
- **F15.** **Daemon distribution** — build and ship `beadle-daemon` the way `beadle-email` already ships — cross-compiled binaries, an install path, a way to run it as a service. Without this, Pipeline v1 and v2 are permanently unreachable by anyone but the author

Should Do

Features for after the Must Do items ship. Not launch-blocking.

- **F16.** **LLM planner** — replace regex planner with Claude decomposition of natural language instructions into command sequences
- **F17.** **Bridge health monitoring** — configurable health checks, a status command, outage logging, and a retry queue for pending result deliveries — see FAQ 13
- **F18.** **Service-manager integration** — a macOS LaunchAgent and a systemd unit that restart the daemon on failure
- **F19.** **Internal scheduler** — cron-like scheduled pipeline execution with no catch-up on missed runs
- **F20.** **Multi-channel triggers** — biff chat, webhooks as trigger channels alongside email
- **F21.** **Key rotation management** — guided key rotation with expiry enforcement
- **F22.** **Multi-provider email** — Fastmail, Gmail, self-hosted Dovecot. Standard IMAP/SMTP, no architectural lock-in

Won't Do

Features explicitly excluded. Naming what you won't build prevents scope creep and clarifies the product's identity.

- **F23.** **Multi-tenancy** — Beadle is a personal agent for one owner — not to be confused with Feature 4's multiple *identities for that one owner*, which is shipped. Serving multiple distinct owners from one daemon adds complexity that conflicts with the single-owner trust model
- **F24.** **Signal or other messaging integrations** — email is the remote interface. Adding messaging platforms fragments the authentication model and increases attack surface
- **F25.** **Web UI** — a browser interface contradicts the local-first, CLI-native design philosophy and adds a large attack surface
- **F26.** **Cloud hosting or SaaS model** — Beadle runs on the owner's machine. A hosted version would undermine the core value proposition of data sovereignty and local control
- **F27.** **Scheduler catch-up** — missed scheduled runs are skipped, not replayed. Catch-up logic adds complexity and ambiguity about which version of a command should execute

F28. **Expired or non-expiring command-signing keys** — all command-signing keys must carry an expiration date that has not yet passed. This is a security invariant, not a convenience trade-off

References

[1] CNBC. *From Clawdbot to Moltbot to OpenClaw: Meet the AI Agent Generating Buzz and Fear Globally*. Reports 145,000+ GitHub stars and 20,000 forks as of 2026-02-02, the commonly-cited viral-growth milestone. 2026. URL: https://www.cnbc.com/2026/02/02/openclaw-open-source-ai-agent-rise-controversy-clawdbot-moltbot-moltbook.html.

[2] Kaspersky. *OpenClaw/ClawdBot Vulnerabilities*. Authentication disabled by default; three CVEs including CVSS 8.8 gateway compromise. 2026. URL: https://securelist.com/openclaw-clawdbot-vulnerabilities/115847/.

[3] SOCRadar. *CVE-2026-25253: 1-Click RCE in OpenClaw Through Auth Token Exfiltration*. Primary technical writeup of the gatewayUrl/WebSocket-token-exfiltration RCE chain, CVSS 8.8. 2026. URL: https://socradar.io/blog/cve-2026-25253-rce-openclaw-auth-token/.

[4] Snyk. *ClawdBot Security Analysis: CVE-2026*. Demonstrated spoofed email triggering shell command that exfiltrated clawdbot.json containing API keys. 2026. URL: https://snyk.io/blog/clawdbot-security-analysis-cve-2026/.

[5] HumanLayer. *ACP: Agent Control Plane*. Open-source agent scheduler with email as a human-approval channel for outer-loop agents. 2024. URL: https://github.com/humanlayer/agentcontrolplane.

[6] Proton. *Proton Mail Bridge*. Local IMAP/SMTP server; open-source Go; paid subscription required. 2025. URL: https://proton.me/mail/bridge.

[7] LangChain. *State of Agent Engineering*. Survey of 1,340 practitioners, Nov 18–Dec 2, 2025; 57.3% have agents in production, up from 51% the prior year; supersedes/updates langchain2024agents production figure. 2025. URL: https://www.langchain.com/state-of-agent-engineering.

[8] LangChain. *State of AI Agents Report: 2024 Trends*. Survey of 1,300+ professionals; 51% using agents in production. 2024. URL: https://www.langchain.com/stateofaiagents.

[9] Significant Gravitas. *AutoGPT*. 187,000+ GitHub stars as of 2026-08; autonomous AI agent framework with local deployment. 2024. URL: https://github.com/Significant-Gravitas/AutoGPT.

[10] Cisco. *Cisco's 2025 Data Privacy Benchmark Study*. 2,600 professionals across 12 countries; 64% worry about inadvertently sharing sensitive info publicly/with competitors via GenAI (not cloud-specific); 90% see local storage as safer; 91% (up 5pp YoY) trust global providers for better data protection. 2025. URL: https://newsroom.cisco.com/c/r/newsroom/en/us/a/y2025/m04/cisco-2025-data-privacy-benchmark-study-privacy-landscape-grows-increasingly-complex-in-the-age-of-ai.html.

[11] OWASP GenAI Security Project. *OWASP Top 10 Risks and Mitigations for Agentic AI Security*. 100+ security researchers; covers prompt injection, identity abuse, tool misuse for agentic AI. 2025. URL: https://genai.owasp.org/2025/12/09/owasp-genai-security-project-releases-top-10-risks-and-mitigations-for-agentic-ai-security/.

[12] Proton. *Proton Reaches 100 Million Accounts*. 100M+ accounts; demonstrates demand for privacy-focused communication. 2024. URL: https://proton.me/blog/proton-100-million-accounts.

[13] Ivar Jacobson and Alistair Cockburn. "Use Cases are Essential". In: *ACM Queue* 21.5 (2023). ACM peer-reviewed; argues for renewed use of use case methodology, pp. 66–86. URL: https://dl.acm.org/doi/fullHtml/10.1145/3631182.

[14] hoop.dev. *GPG Security Review: Strengths, Weaknesses, and Best Practices*. Key expiry and rotation as required best practice; complexity as primary operational risk. 2024. URL: https://hoop.dev/blog/gpg-security-review-strengths-weaknesses-and-best-practices/.

[15] Werner Koch et al. *RFC 9580: OpenPGP*. Current OpenPGP standard; July 2024; v6 formats, post-quantum ML-KEM keys. 2024. URL: https://www.rfc-editor.org/rfc/rfc9580.html.

[16] The Hacker News. *OpenClaw Bug Enables One-Click Remote Code Execution via Malicious Link*. Corroborates CVE-2026-25253 attack chain; disclosed 2026-01-26, patched in v2026.1.29. 2026. URL: https://thehackernews.com/2026/02/openclaw-bug-enables-one-click-remote.html.

[17] OpenClaw / GitHub Security Advisory. *GHSA-mc68-q9jw-2h3v: Command Injection in Clawdbot Docker Execution via PATH Environment Variable*. Official advisory for CVE-2026-24763: authenticated command injection (CWE-78) via unsafe PATH handling in Docker sandbox execution, unrelated to the skill directory. 2026. URL: https://github.com/openclaw/openclaw/security/advisories/GHSA-mc68-q9jw-2h3v.

[18] Oren Yomtov and Koi Security. *Analysis of ClawHub Malicious Skills Poisoning (ClawHavoc)*. Audit of all 2,857 ClawHub skills found 341 malicious (11.9%); 335 traced to the ClawHavoc campaign distributing AMOS infostealer malware to an estimated 300,000 OpenClaw users. 2026. URL: https://www.esecurityplanet.com/threats/hundreds-of-malicious-skills-found-in-openclaws-clawhub/.

[19] Palo Alto Networks Unit 42. *OpenClaw Agentic AI Threat Assessment*. Concluded OpenClaw maps to all 10 OWASP risks for agentic applications; lacks enforceable trust boundaries between untrusted inputs and high-privilege reasoning. 2026. URL: https://unit42.paloaltonetworks.com/openclaw-agentic-ai-threat-assessment/.